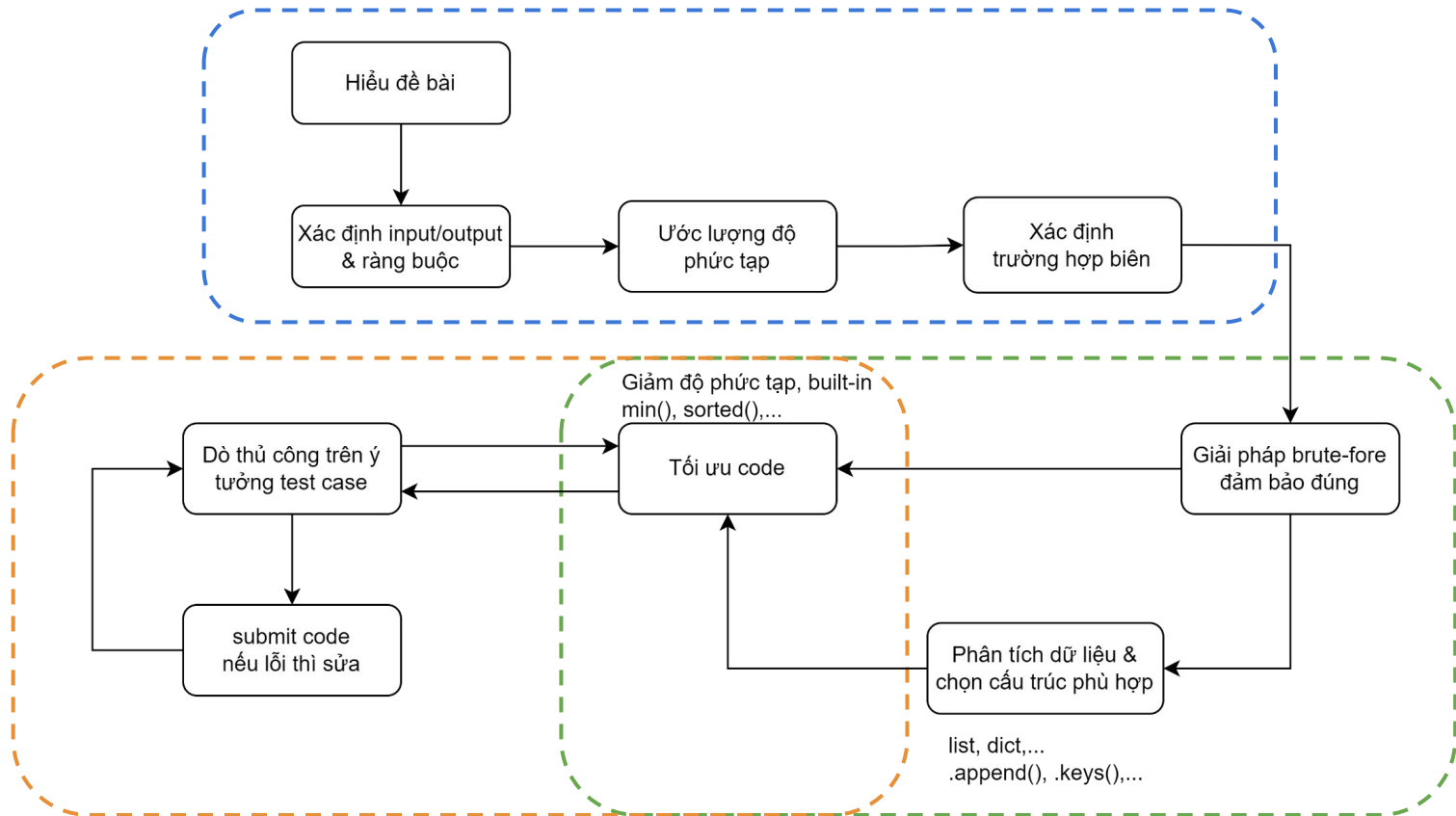


BUỔI 3: BÀI TẬP
15/11/2025

CÁCH TIẾP CẬN



CÁCH TIẾP CẬN

1. Hiểu rõ đề bài:

- Bài toán có đầy đủ dữ kiện không?
- Có thể chia bài toán thành những phần nhỏ hơn không?
- Từ khóa quan trọng? (*đếm, sắp xếp, tìm, lọc, ghép cặp...*)

2. Xác định Input/Output và ràng buộc

- Có bao nhiêu Input, định dạng gì?
- Output là gì, định dạng? (*câu trả lời, danh sách, tổng, chuỗi, v.v.*)
- Dữ liệu có thể trống, âm, hoặc trùng lặp không?

→ **Gợi ý:** Đọc kỹ ví dụ và mô tả input/output để tránh hiểu sai đề.

CÁCH TIẾP CẬN

3. Ước lượng độ phức tạp

- **Mục tiêu:** Biết chương trình của mình chạy nhanh hay chậm, có quá tốn bộ nhớ không
- Dựa vào trực giác, kinh nghiệm
 - + Nếu vòng lặp chồng vòng lặp → giảm bớt lặp lại.
 - + Nếu tìm kiếm nhiều lần → sắp xếp trước hoặc dùng dict / set.
 - + Nếu dữ liệu quá lớn → xử lý từng phần
- Cần nhớ:
 - + “Càng nhiều vòng lặp lồng nhau → càng chậm.”
 - + “Dữ liệu càng lớn → càng cần chọn cách tính gọn hơn.”

CÁCH TIẾP CẬN

4. Xác định trường hợp biên (edges case)

- Edge case = trường hợp “**đặc biệt**” mà đề bài không nói rõ, nhưng **code của bạn phải xử lý đúng**
- **Ví dụ:** Kiểm tra chuỗi có phải là ngày hợp lệ theo chuẩn Gregorian, định dạng đúng **YYYY-MM-DD** hay không?
 - + normal cases: “2025-11-15”
 - + trường hợp biên:
 - Tháng 2 giới hạn: “2025-02-29”
 - Năm nhuận: “2024-02-29” (biết quy tắc tính năm nhuận)
 - Năm biên: 0000-01-01 → (biết quy ước lịch Gregorian không có năm 0).
 -

CÁCH TIẾP CẬN

5. Giải pháp brute-force (đảm bảo đúng)

- Brute-force = Cách làm “thô sơ” nhưng chắc chắn đúng → Có lời giải đúng trước, nhanh chậm tính sau
- Duyệt hết mọi khả năng, không cần tối ưu lúc đầu.
- Giúp kiểm chứng ý tưởng trước khi tối ưu.
- **Ví dụ:** Tìm 3 số có tổng lớn nhất trong danh sách.
 - + → Duyệt mọi cặp (i, j, k) → Đảm bảo ra kết quả đúng, dù chậm.

```
nums = [1, 6, 3, 4, 8, 9, 1, 4, 6, 10, 17, 6, 5, 2]
n = len(nums)
for i in range(n):
    for j in range(i + 1, n):
        for k in range(j + 1, n):
            s = nums[i] + nums[j] + nums[k]
            if s > max_sum:
                max_sum = s
```

số tổ hợp chọn 3 phần tử từ n phần tử:
 $n \times (n-1) \times (n-2) / 6$ tổ hợp

- 100 → 161,700
- 1000 → 166,167,000

CÁCH TIẾP CẬN

6. Phân tích dữ liệu & chọn CTDL

- Chọn đúng cấu trúc giúp code ngắn hơn, chạy nhanh hơn, và dễ debug hơn.
→ “Hãy để cấu trúc dữ liệu phục vụ cho cách xử lý — không phải ngược lại.”

Câu hỏi	Gợi ý chọn cấu trúc
Dữ liệu chỉ là danh sách các phần tử?	→ <code>list</code>
Cần gán tên (key) cho từng giá trị?	→ <code>dict</code>
Cần loại bỏ trùng lặp ?	→ <code>set</code>
Cần giữ thứ tự cố định ?	→ <code>list</code> hoặc <code>tuple</code>
Dữ liệu dạng cặp (key, value) cố định?	→ <code>tuple</code>

CÁCH TIẾP CẬN

6. Phân tích dữ liệu & chọn CTDL

- **Ví dụ 1:** Nhập vào danh sách điểm của sinh viên trong lớp, sau đó in ra điểm trung bình.

→ list

```
scores = [8, 6.5, 9, 7.5, 10]      # list chứa các điểm
average = sum(scores) / len(scores)
print("Điểm trung bình:", average)
```

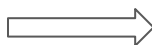
CÁCH TIẾP CẬN

6. Phân tích dữ liệu & chọn CTDL

- **Ví dụ 2:** Lưu điểm của từng sinh viên và in ra điểm của người có tên "An".

→ dict

```
scores = {"An": 8.5, "Bình": 7, "Chi": 9}
print("Điểm của An:", scores["An"])
```



Nếu lớp có 2 sinh viên trùng tên nhau?



```
students = {
    1001: {"name": "An", "score": 8.5},
    1002: {"name": "An", "score": 9.0},
    1002: {"name": "Bình", "score": 10.0},
}
# Tạo danh sách các bạn tên "An"
result = []
for sid, info in students.items():
    if info["name"] == "An":
        result.append((sid, info["score"]))

print("Các bản ghi tên An:")
for sid, sc in result:
    print(f"- ID {sid}: {sc}")
```

```
scores = {
    "An": [8.5, 9.0], # có 2 bạn
    tên An
    "Bình": [7.0]
}
→ Không có định danh, tên có thể trùng
⇒ dict key: tên, value: list điểm
```

CÁCH TIẾP CẬN

7. Tối ưu code

- Code này có chạy đúng với mọi trường hợp, kể cả edge case chưa?
- Có hướng giải khác nhanh hơn không?
- Hiệu năng có thể cải thiện thêm không?
- Làm sao để code dễ đọc hơn?
- Có bước nào hoặc hàm nào dư thừa không?
- Có đoạn lặp nào có thể bỏ bớt hoặc gom lại không?

→ Giảm số vòng lặp và lồng nhau

→ Chọn cấu trúc dữ liệu đúng (tra cứu nhanh dùng dict/set).

→ Tránh tính đi tính lại (lưu kết quả tạm, “ghi nhớ”).

→ Dừng sớm khi đã đủ (early return, break).

→ Dùng hàm dựng sẵn: sum, max, min, sorted, any, all (thường tối ưu hơn tự viết vòng lặp).

CÁCH TIẾP CẬN

7. Tối ưu code

- Ví dụ: Tìm 3 số có tổng lớn nhất trong danh sách.

Brute-force

```
nums = [1,6.3,3,4,8,9.9,1,4,6.4,10,17,6,5.2,2]
```

```
n = len(nums)
```

```
for i in range(n):
```

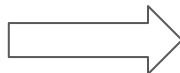
```
    for j in range(i + 1, n):
```

```
        for k in range(j + 1, n):
```

```
            s = nums[i] + nums[j] + nums[k]
```

```
            if s > max_sum:
```

```
                max_sum = s
```



quét 1 lần giữ top3

```
top1 = top2 = top3 = float("-inf")
```

```
for x in arr:
```

```
    if x > top1:
```

```
        top1, top2, top3 = x, top1, top2
```

```
    elif x > top2:
```

```
        top2, top3 = x, top2
```

```
    elif x > top3:
```

```
        top3 = x
```

```
max_sum = top1 + top2 + top3
```

CÁCH TIẾP CẬN

8. Dò thủ công trên ý tưởng test case

- Giúp phát hiện lỗi logic, hiểu rõ dòng chảy xử lý.
- Chọn test case tiêu biểu: TH đơn giản, TH biên
- Giả lập bằng tay từng bước thuật toán → thay đổi từng bước, so sánh kết quả với dự đoán
- Đánh dấu bước sai nếu kết quả lệch → xem lại phân tích logic, thuật toán.

CÁCH TIẾP CẬN

8. Dò thủ công trên ý tưởng test case

- arr = [5, 1, 7, 3, 9, 2]

Brute-force

n = len(nums)

for i in range(n):

 for j in range(i + 1, n):

 for k in range(j + 1, n):

 s = nums[i] + nums[j] + nums[k]

 if s > max_sum:

 max_sum = s

#	(i,j,k)	Bộ ba (a,b,c)	Tổng s	Cập nhật?	max_sum sau bước	#	(i,j,k)	Bộ ba (a,b,c)	Tổng s	Cập nhật?	max_s um sau bước
1	(0,1,2)	(5,1,7)	13	ok	13	10	(0,4,5)	(5,9,2)	16	x	21
2	(0,1,3)	(5,1,3)	9	x	13	11	(1,2,3)	(1,7,3)	11	x	21
3	(0,1,4)	(5,1,9)	15	ok	15	12	(1,2,4)	(1,7,9)	17	x	21
4	(0,1,5)	(5,1,2)	8	x	15	13	(1,2,5)	(1,7,2)	10	x	21
5	(0,2,3)	(5,7,3)	15	x	15	14	(1,3,4)	(1,3,9)	13	x	21
6	(0,2,4)	(5,7,9)	21	ok	21	15	(1,3,5)	(1,3,2)	6	x	21
7	(0,2,5)	(5,7,2)	14	x	21	16	(1,4,5)	(1,9,2)	12	x	21
8	(0,3,4)	(5,3,9)	17	x	21	17	(2,3,4)	(7,3,9)	19	x	21
9	(0,3,5)	(5,3,2)	10	x	21	18	(2,3,5)	(7,3,2)	12	x	21
						19	(2,4,5)	(7,9,2)	18	x	21
						20	(3,4,5)	(3,9,2)	14	x	21

CÁCH TIẾP CẬN

8. Dò thủ công trên ý tưởng test case

- arr = [5, 1, 7, 3, 9, 2]

quét 1 lần giữ top3

```
top1 = top2 = top3 = float("-inf")
```

```
for x in arr:
```

```
    if x > top1:
```

```
        top1, top2, top3 = x, top1, top2
```

```
    elif x > top2:
```

```
        top2, top3 = x, top2
```

```
    elif x > top3:
```

```
        top3 = x
```

```
max_sum = top1 + top2 + top3
```

Kết quả mong đợi:

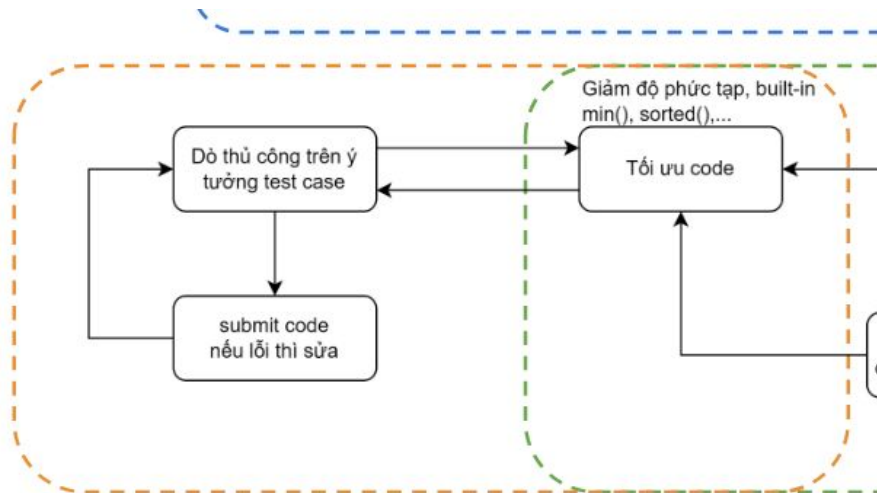
top1 = 9, top2 = 7, top3 = 5

→ max_sum = 21

Bước	x	top1	top2	top3	Ghi chú
1	5	5	-inf	-inf	khởi tạo top1
2	1	5	1	-inf	x > top2
3	7	7	5	1	x > top1 → dồn
4	3	7	5	3	x > top3
5	9	9	7	5	x > top1 → dồn
6	2	9	7	5	không thay đổi

CÁCH TIẾP CẬN

8. Nộp Bài



Thực hành

Expressions:

[13.6 PRACTICE: Expressions*: Simple floating-point expression](#)

[13.9 PRACTICE: Expressions* Proper name layout](#)

Branches

[13.13 PRACTICE: Branches*: Listing names](#)

[13.14 PRACTICE: Branches**: Complex cost structure](#)

Loops

[13.24 PRACTICE: Loops*: Password requirements](#)

[13.27 PRACTICE: Loops \(while\)**: Output rocket height](#)

List:

[13.58 PRACTICE: Lists**: Min, max, average](#)

Function:

[13.41 PRACTICE: Functions*: Rotate three values](#)